Ε.Μ.Π. - ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ. ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ
ΑΚΑΔ. ΕΤΟΣ 2024-2025

ΑΘΗΝΑ 15 - 05 - 2026

2η ΟΜΑΔΑ ΑΣΚΗΣΕΩΝ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Συστήματα Μικροϋπολογιστών"

Παράδοση 31/05/2026
Ασκήσεις προσομοίωσης της αρχιτεκτονικής ARM

Όπου ζητείται να γίνει χρήση του **STM32CubeIDE** με το πρόγραμμα προσομοίωσης **Renode** και το **GUI tool**

1η ΑΣΚΗΣΗ: Δίνεται το παρακάτω πρόγραμμα σε **hex machine code** για μικροελεγκτή **ARM Cortex-M4** (STM32F407VGT6):

```
0A 20 03 21 82 00 52 18 05 21 52 1A 01 4B 1A 60 FE E7 00 00 00 10 00 20
```

(α) Να γίνει disassembly του παραπάνω κώδικα μηχανής και να γραφεί η αντίστοιχη ARM Thumb-2 assembly.

(β) Να εξηγηθεί τι υπολογίζει το πρόγραμμα.

(γ) Να εκτελεστεί το πρόγραμμα με το εργαλείο προσομοίωσης και να επιβεβαιωθεί η λειτουργία του.

Παρατήρηση: Ο κώδικας μηχανής δεν περιέχει vector tables και offsets.

2η ΑΣΚΗΣΗ: Έστω ότι έχουμε στην διάθεσή μας Σύστημα σε Ψηφίδα (SoC) βασισμένο στον μικροελεγκτή **ARM Cortex-M4** (STM32F407VGT6) με το **GPIOA (PA) ως input** και το **GPIOD (PD) ως output**. Να γράφει σε assembly πρόγραμμα το οποίο να απεικονίζει ένα αναμμένο led το οποίο να κινείται αριστερά στο **GPIOA** (από το LSB προς το MSB) και να συνεχίζει να κινείται κυκλικά (θέσεις led 0,1,2,3,4,5,…,14,15,0,1,2 κ.λπ.) όταν **το MSB της θύρας GPIOD** είναι OFF. Αλλιώς, **όταν το MSB της θύρας GPIOD** γίνεται ON να αναστρέφεται η κατεύθυνση και το αναμμένο led να κινείται δεξιά (από το MSB προς το LSB). Τέλος, **όταν το LSB της θύρας GPIOD** γίνεται OFF το led να σταματάει εκεί που βρίσκεται. Στη συνέχεια, όταν ξαναγίνει ON να συνεχίζεται η κίνησή του σύμφωνα με το MSB της θύρας GPIOD. Αρχική θέση του αναμμένου led **να θεωρηθεί το LSB**. Να επαληθευτεί η λειτουργία της ζητούμενης υλοποίησης στο πρόγραμμα προσομοίωσης Renode με το GUI tool.

3η ΑΣΚΗΣΗ: Να γραφεί πρόγραμμα σε ARM assembly για μικροελεγκτή ARM Cortex-M4 (STM32F407VGT6) που εκτελεί τις παρακάτω λειτουργίες στο πρόγραμμα προσομοίωσης Renode με το GUI tool. Θεωρήστε ως αρχική διεύθυνση δεδομένων στη SRAM τη διεύθυνση 0x20000900.

(α) Να αποθηκευτούν οι αριθμοί 0-255 **με φθίνουσα σειρά** στις διαδοχικές θέσεις της μνήμης με αρχή τη διεύθυνση 0x20000900. Οι αριθμοί να αποθηκευτούν **όχι χειροκίνητα αλλά μέσω προγράμματος** και να ελέγξετε αν έγινε η ζητούμενη λειτουργία σωστά. Κάθε αριθμός να αποθηκευτεί σε **μέγεθος λέξης**.
(β) Υπολογίστε τον αριθμό **των μηδενικών** (το πλήθος των δυαδικών ψηφίων 0) των παραπάνω 256 δεδομένων. Το αποτέλεσμα να εμφανιστεί στο GPIOD.
(γ) Υπολογίστε το πλήθος από τους παραπάνω αριθμούς (0-255) που είναι μεταξύ των αριθμών $0x10 \leq x \leq 0x80$. Το αποτέλεσμα να εμφανιστεί επίσης στο **GPIOD**.
(δ) Τι διαφορετικό παρατηρείτε σε σύγκριση με την αντίστοιχη **άσκηση 1 για τον AVR (1η Σειρά Ασκήσεων)**;

Γενική Παρατήρηση 1: *Τα προγράμματα να συνοδεύονται υποχρεωτικά στα κυριότερα σημεία τους από σύντομα σχόλια.*
Γενική Παρατήρηση 2: *Για επαλήθευση της ορθής λειτουργίας των προγραμμάτων, να συμπεριλάβετε ευκρινή screenshots επιλεγμένων σεναρίων, τα οποία πρέπει να συνοδεύονται υποχρεωτικά με την αντίστοιχη επεξήγηση του σεναρίου.*

Θεωρητικές Ασκήσεις

4η ΑΣΚΗΣΗ: Σας δίνονται 4 use cases (εφαρμογές) στις οποίες θα χρησιμοποιηθεί ενσωματωμένο σύστημα με επεξεργαστή αρχιτεκτονικής ARM. Οι εφαρμογές είναι οι εξής:

- **Automotive systems controller:** Πίεση ελαστικών, αερόσακοι, έλεγχος επαφής με έδαφος. Εξαιρετικά υψηλή ταχύτητα απόκρισης.
- **Drone path planning with ROS (Robot Operating System): 2 tasks** – (α) Μετρήσεις από αισθητήρες κίνησης και θέσης, (β) Λειτουργικό Σύστημα (ROS) με multi-threading
- **Weather Base Station:** Έλεγχος αισθητήρων θερμοκρασίας, υγρασίας και φωτός κάθε 10 λεπτά σε απομακρυσμένο εξωτερικό χώρο. Υψηλή αυτονομία.
- **IoT for Health:** Καταγραφή ενδείξεων ζωτικής λειτουργίας και παραγωγή στατιστικών. Ενσωματωμένος σε φορητή συσκευή με σχετικά υψηλή αυτονομία.

(α) Επιλέξτε έναν από τους παρακάτω ARM επεξεργαστές για κάθε εφαρμογή: **Cortex A15, Cortex R5, Cortex M4, Cortex M0**. Αιτιολογείστε την απάντηση σας σχετικά με τις απαιτήσεις κάθε εφαρμογής.

(β) Σε ποιες από τις παραπάνω εφαρμογές θα εξυπηρετούσε μία οργάνωση big.Little και γιατί;

5η ΑΣΚΗΣΗ: Εκτελέστε με το χέρι τον παρακάτω κώδικα ARM assembly σε AArch32 για επεξεργαστή ARMv7-A και κρατήστε την κατάσταση των **CPSR, SP και PC** (βλ. διάλεξη 15 - "ARM Programmers Model" διαφάνειες 15-17), με δεδομένο ότι η αρχική τους κατάσταση είναι η παρακάτω:

```
PC = 0x00008000
SP = 0x00010000
r0 = 0x00000000
r1 = 0x00000000
r2 = 0x00000000
r3 = 0x00000000
r4 = 0x00000000
r5 = 0x00000000
r9 = 0x00000000
CPSR = 0x00000013
```

Δίνεται ο κώδικας προς εκτέλεση:

```
        .syntax unified
        .arch armv7-a
        .text
        .global _start
        .arm ; Entering aarch32 mode
_start:
        MOV     r0, #0x80000000
        ADDS    r1, r0, r0

CHK1:
        STMDB   sp!, {r0, r1}

CHK2:
        MOV     r2, #0
        CMP     r2, #1

        BLT     to_thumb
        MOV     r9, #9

to_thumb:
        ADR     r3, thumb_part + 1
        BX      r3

        .thumb ; Entering Thumb2 mode
        .thumb_func
thumb_part:
CHK3:
        CMP     r2, #0
        POP     {r4, r5}

        ADR     r3, arm_back
        BX      r3

        .align 2
        .arm ; Entering aarch32 mode
arm_back:
CHK4:
        B       CHK4
```

Στα 4 checkpoints (CHK1, CHK2, CHK3 και CHK4) που σας δίνονται εντός του κώδικα, να γράψετε ακριβώς την κατάσταση των **CPSR, SP, PC και r0-r9**.